Sliding Window Technique Study Guide

1. Introduction to Sliding Window

The Sliding Window Technique is an algorithmic approach used to efficiently solve problems that involve arrays or strings. Instead of using nested loops that can be inefficient, this technique maintains a "window" of elements and slides it across the data structure.

How It Works

The sliding window technique works by:

- Creating a window of a specific size or dynamic size
- Moving the window from left to right across the array/string
- Updating the window contents as it moves
- Keeping track of the optimal solution found so far

Types of Sliding Window

Fixed Size Window:

- The window size remains constant throughout the process
- Used when the problem specifies a fixed window size

Variable Size Window:

- The window size can expand or contract based on certain conditions
- Used when we need to find optimal subarray/substring of any size

Basic Template

```java
// Fixed Size Sliding Window
public int fixedSizeWindow(int[] arr, int k) {
    int windowSum = 0;
    int maxSum = 0;

    // Calculate sum of first window
    for (int i = 0; i < k; i++) {
        windowSum += arr[i];
    }
    maxSum = windowSum;

    // Slide the window
    for (int i = k; i < arr.length; i++) {
        windowSum = windowSum - arr[i - k] + arr[i];
        maxSum = Math.max(maxSum, windowSum);
    }

    return maxSum;
}
```

```java
// Variable Size Sliding Window
public int variableSizeWindow(int[] arr, int target) {
    int left = 0, right = 0;
    int windowSum = 0;
    int result = 0;

    while (right < arr.length) {
        // Expand window
        windowSum += arr[right];

        // Contract window if needed
        while (windowSum > target && left <= right) {
            windowSum -= arr[left];
            left++;
        }

        // Update result
        if (windowSum == target) {
            result = Math.max(result, right - left + 1);
        }

        right++;
    }

    return result;
}
```

2. Use Cases of Sliding Window

Maximum Sum Subarray of Size K

Find the maximum sum of any contiguous subarray of size k.

```java
public int maxSumSubarray(int[] arr, int k) {
    if (arr.length < k) return -1;

    int windowSum = 0;

    // Sum of first window
    for (int i = 0; i < k; i++) {
        windowSum += arr[i];
    }

    int maxSum = windowSum;

    // Slide the window
    for (int i = k; i < arr.length; i++) {
        windowSum = windowSum - arr[i - k] + arr[i];
        maxSum = Math.max(maxSum, windowSum);
    }

    return maxSum;
}
```

Longest Substring Without Repeating Characters

Find the length of the longest substring without repeating characters.

```java
import java.util.*;

public int longestUniqueSubstring(String s) {
    Set<Character> window = new HashSet<>();
    int left = 0, maxLength = 0;

    for (int right = 0; right < s.length(); right++) {
        // Contract window until no duplicates
        while (window.contains(s.charAt(right))) {
            window.remove(s.charAt(left));
            left++;
        }

        // Add current character to window
        window.add(s.charAt(right));

        // Update maximum length
        maxLength = Math.max(maxLength, right - left + 1);
    }

    return maxLength;
}
```

Minimum Window Substring

Find the minimum window in string s that contains all characters of string t.

```java
import java.util.*;

public String minWindow(String s, String t) {
    Map<Character, Integer> targetMap = new HashMap<>();
    for (char c : t.toCharArray()) {
        targetMap.put(c, targetMap.getOrDefault(c, 0) + 1);
    }

    int left = 0, minLen = Integer.MAX_VALUE;
    int minStart = 0, matched = 0;
    Map<Character, Integer> windowMap = new HashMap<>();

    for (int right = 0; right < s.length(); right++) {
        char rightChar = s.charAt(right);
        windowMap.put(rightChar, windowMap.getOrDefault(rightChar, 0) + 1);

        if (targetMap.containsKey(rightChar) &&
            windowMap.get(rightChar).equals(targetMap.get(rightChar))) {
            matched++;
        }

        // Contract window
        while (matched == targetMap.size()) {
            if (right - left + 1 < minLen) {
                minLen = right - left + 1;
                minStart = left;
            }

            char leftChar = s.charAt(left);
            if (targetMap.containsKey(leftChar) &&
                windowMap.get(leftChar).equals(targetMap.get(leftChar))) {
                matched--;
            }
            windowMap.put(leftChar, windowMap.get(leftChar) - 1);
            left++;
        }
    }

    return minLen == Integer.MAX_VALUE ? "" : s.substring(minStart, minStart + minLen);
}
```

Subarray with Given Sum

Find if there exists a contiguous subarray with sum equal to target.

```java
public boolean subarraySum(int[] arr, int target) {
    int left = 0, currentSum = 0;

    for (int right = 0; right < arr.length; right++) {
        currentSum += arr[right];

        // Contract window if sum exceeds target
        while (currentSum > target && left <= right) {
            currentSum -= arr[left];
            left++;
        }

        if (currentSum == target) {
            return true;
        }
    }

    return false;
}
```

3. Practice Questions on Sliding Window Technique

Example 1: Average of Subarrays of Size K

```java
import java.util.*;

public double[] findAverages(int[] arr, int k) {
    double[] result = new double[arr.length - k + 1];
    double windowSum = 0;
    int windowStart = 0;

    for (int windowEnd = 0; windowEnd < arr.length; windowEnd++) {
        windowSum += arr[windowEnd];

        if (windowEnd >= k - 1) {
            result[windowStart] = windowSum / k;
            windowSum -= arr[windowStart];
            windowStart++;
        }
    }

    return result;
}
```

Example 2: Maximum Number of Vowels in Substring of Length K

```java
public int maxVowels(String s, int k) {
    Set<Character> vowels = Set.of('a', 'e', 'i', 'o', 'u');
    int maxVowelCount = 0, currentVowelCount = 0;

    // Count vowels in first window
    for (int i = 0; i < k; i++) {
        if (vowels.contains(s.charAt(i))) {
            currentVowelCount++;
        }
    }
    maxVowelCount = currentVowelCount;

    // Slide the window
    for (int i = k; i < s.length(); i++) {
        // Remove leftmost character
        if (vowels.contains(s.charAt(i - k))) {
            currentVowelCount--;
        }
        // Add rightmost character
        if (vowels.contains(s.charAt(i))) {
            currentVowelCount++;
        }
        maxVowelCount = Math.max(maxVowelCount, currentVowelCount);
    }

    return maxVowelCount;
}
```

Example 3: Longest Subarray with At Most K Distinct Characters

```java
import java.util.*;

public int longestSubstringKDistinct(String s, int k) {
    if (k == 0) return 0;

    Map<Character, Integer> charFrequency = new HashMap<>();
    int windowStart = 0, maxLength = 0;

    for (int windowEnd = 0; windowEnd < s.length(); windowEnd++) {
        char rightChar = s.charAt(windowEnd);
        charFrequency.put(rightChar, charFrequency.getOrDefault(rightChar, 0) + 1);

        // Contract window if we have more than k distinct characters
        while (charFrequency.size() > k) {
            char leftChar = s.charAt(windowStart);
            charFrequency.put(leftChar, charFrequency.get(leftChar) - 1);
            if (charFrequency.get(leftChar) == 0) {
                charFrequency.remove(leftChar);
            }
            windowStart++;
        }

        maxLength = Math.max(maxLength, windowEnd - windowStart + 1);
    }

    return maxLength;
}
```

Quiz Questions

Question 1: What is the main advantage of using the sliding window technique? a) It reduces space complexity b) It reduces time complexity from $O(n^2)$ to $O(n)$ c) It makes code easier to read d) It works only with sorted arrays

Answer: b) It reduces time complexity from $O(n^2)$ to $O(n)$

Question 2: In a fixed-size sliding window of size k, what operation is performed when moving the window one position to the right? a) Add the new rightmost element only b) Remove the leftmost element only c) Remove the leftmost element and add the new rightmost element d) Recalculate the entire window sum

Answer: c) Remove the leftmost element and add the new rightmost element

Question 3: Which type of sliding window is best suited for finding the "longest substring with at most k distinct characters"? a) Fixed-size window b) Variable-size window c) Both work equally well d) Neither, this problem cannot use sliding window

Answer: b) Variable-size window

Question 4: In the variable-size sliding window technique, when do we typically expand the window? a) When the current window satisfies our condition b) When the current window violates our condition c) Always, until we reach the end of the array d) Only when the window is empty

Answer: c) Always, until we reach the end of the array

Question 5: What data structure is commonly used alongside sliding window technique to track character frequencies? a) Array b) Stack c) HashMap d) Queue

Answer: c) HashMap

Practice Coding Questions

Question 1: Write a Java method to find the smallest subarray with sum greater than or equal to a given target value. Return the length of such subarray. If no such subarray exists, return 0.

```java
public int minSubArrayLen(int target, int[] nums) {
    // Your code here
}
```

Question 2: Given a string, find the length of the longest substring that contains at most 2 distinct characters.

```java
public int lengthOfLongestSubstringTwoDistinct(String s) {
    // Your code here
}
```

Question 3: Find the maximum sum of any contiguous subarray of size k. If the array has fewer than k elements, return -1.

```java
public int maxSumSubarrayOfSizeK(int[] arr, int k) {
    // Your code here
}
```

Question 4: Given a binary array and an integer k, return the maximum number of consecutive 1's in the array if you can flip at most k 0's.

```java
public int longestOnes(int[] nums, int k) {
    // Your code here
}
```

Question 5: Write a method to find all anagrams of a pattern string in a given text string. Return the starting indices of all anagrams.

```java
import java.util.*;
public List<Integer> findAnagrams(String s, String p) {
    // Your code here
}
```